

Proceso de aprendizaje 6

PROGRAMACIÓN WEB II

Proyecto integrador full stack, despliegue y seguridad

Curso	Programación Web II
Código	30690
Periodo académico	202610
Proceso de evaluación	Proceso de evaluación - 6
Sesión	16
Peso en el curso	15%
Modalidad	Trabajo grupal: 4 estudiantes, máximo 5
Calificación	Escala vigesimal: 0 a 20 puntos

Proyecto integrador sugerido

Plataforma de Gestión de Cursos e Inscripciones. El equipo puede conservar el mismo dominio desarrollado y propuesto a inicio de clase, siempre que cumpla todos los requerimientos técnicos y funcionales de esta evaluación.

1. Propósito de la evaluación

Demostrar la competencia final del curso mediante el diseño, integración, documentación, despliegue y sustentación de una aplicación web moderna. La solución deberá articular frontend, backend, persistencia, autenticación, gestión de estado, optimización para producción y controles básicos de seguridad.

2. Reto integrador

El equipo deberá presentar una versión final funcional del proyecto construido durante el curso. La aplicación debe resolver un flujo de negocio de principio a fin y permitir que un usuario autenticado interactúe con datos persistidos, mientras que un usuario con rol administrativo gestiona la información principal del sistema.

Flujo mínimo demostrable

Registro o inicio de sesión → consulta de cursos → inscripción del estudiante → visualización de inscripciones → administración de cursos/usuarios → persistencia en MongoDB Atlas → acceso mediante aplicaciones desplegadas.

3. Alcance funcional obligatorio

- Registro e inicio de sesión de usuarios con validaciones y mensajes de error comprensibles.

- Autenticación mediante JWT y autorización basada en, como mínimo, los roles administrador y estudiante.
- Catálogo o listado de cursos con consulta, búsqueda o filtrado básico.
- Detalle de curso y operación de inscripción del estudiante.
- Panel del estudiante con visualización de sus inscripciones o información asociada.
- Panel administrativo con CRUD funcional de cursos y, al menos, una segunda entidad del sistema.
- Persistencia real de los datos mediante MongoDB y Mongoose.
- Manejo de errores del cliente y del servidor, incluyendo respuestas HTTP coherentes.
- Cierre de sesión y protección de vistas o rutas según el rol.

4. Requerimientos técnicos por capa

Capa / tecnología	Requerimiento mínimo	Evidencia esperada
TypeScript y Angular	Panel administrativo con componentes, servicios, rutas, formularios reactivos, validaciones y consumo de la API.	CRUD administrativo funcionando y código tipado/organizado.
Node.js, Express y MongoDB	API REST modular con rutas, controladores, modelos Mongoose, middlewares, validaciones, JWT, bcrypt y roles.	Endpoints probados y datos persistidos en MongoDB Atlas.
React	Portal del estudiante como SPA con componentes funcionales, Hooks, React Router, consumo de API y sesión.	Flujos de login, catálogo, inscripción y panel del estudiante.
Context API o Redux	Estado global aplicado a autenticación, usuario, inscripciones u otro estado transversal.	Provider/store integrado y utilizado por más de un componente.
Next.js	Vista pública, catálogo o detalle de curso con rutas y evidencia de SSR/SSG o generación estática cuando corresponda.	URL o módulo funcional con explicación de la estrategia utilizada.
Producción y nube	Frontend en Vercel, backend en Render y base de datos en MongoDB Atlas. Variables de entorno correctamente configuradas.	URLs públicas operativas y sin secretos expuestos en GitHub.
Seguridad y calidad	HTTPS, CORS restringido, Helmet, validación de entradas, protección básica frente a XSS/CSRF y auditoría Lighthouse.	Checklist de seguridad, configuración visible y reporte Lighthouse.

5. Entregables obligatorios

Entregable	Contenido obligatorio	Formato / ubicación
Repositorio GitHub	Código fuente completo, historial de commits, estructura clara, .gitignore y archivo .env.example. Puede ser monorepo o repositorios enlazados.	URL pública o acceso concedido al docente.
README principal	Descripción, problema, objetivos, arquitectura, tecnologías, integrantes, instalación, variables, credenciales de prueba, endpoints, URLs desplegadas, capturas y enlace de YouTube.	README.md en la raíz del repositorio.
Aplicaciones desplegadas	Frontend(s) y backend disponibles para evaluación, usar vercel y render respectivamente. La base de datos debe estar alojada en MongoDB Atlas.	URLs verificables.

Documentación técnica	Diagrama de arquitectura, modelo de datos, colección Postman/Thunder Client, decisiones técnicas y checklist de seguridad.	Carpeta /docs dentro del repositorio.
Auditoría de calidad	Reporte Lighthouse de la aplicación pública y breve análisis de resultados y mejoras aplicadas.	PDF, captura o Markdown en /docs.
Video de exposición	Sustentación grupal de 12 a 15 minutos. Todos los integrantes deben participar y mantener sus cámaras prendidas durante su intervención.	Video en YouTube, público o no listado; enlace dentro del README.

6. Estructura obligatoria del video de exposición

1. Presentación del equipo, problema abordado y alcance de la solución.
2. Explicación breve de la arquitectura y distribución de Angular, React, Next.js, Node.js y MongoDB.
3. Demostración en vivo del flujo completo, utilizando las URLs desplegadas.
4. Explicación de autenticación JWT, roles, persistencia, estado global y consumo de API.
5. Presentación de las medidas de seguridad, variables de entorno y reporte Lighthouse.
6. Conclusiones, dificultades resueltas y aporte concreto de cada integrante.

Regla del video

El enlace de YouTube debe estar colocado dentro del README. Todos los integrantes deben aparecer con la cámara prendida y explicar una parte técnica verificable. El video puede estar configurado como “no listado”.

7. Condiciones de entrega y validación de autoría

- La evaluación se desarrolla en grupos de 4 estudiantes, con un máximo de 5 integrantes.
- El repositorio debe reflejar participación progresiva mediante commits identificables. No se aceptará un único commit final como única evidencia de proceso.
- No se deben publicar contraseñas, cadenas de conexión, tokens ni secretos. Debe entregarse un archivo .env.example.
- El docente podrá solicitar una demostración en vivo, modificar un dato, ejecutar un endpoint o formular preguntas individuales.
- La calificación puede diferenciarse entre integrantes cuando la evidencia de participación, asistencia, autoría o sustentación sea distinta.
- La asistencia y participación en las clases entre semanas se calificará individualmente y representa 3 puntos de la rúbrica.
- La copia no atribuida, la reutilización de un proyecto ajeno o la imposibilidad de explicar el código será tratada según las normas académicas institucionales.

8. Rúbrica de calificación - 20 puntos

Criterio	Puntaje máximo	Logro esperado	Guía de asignación
Integración funcional del proyecto	4	El flujo principal funciona de extremo a extremo y responde al caso de negocio.	4: completo y estable. 3: funciona con fallas menores. 2: integración parcial. 0-1: no demuestra el flujo.
Backend, persistencia y autenticación	3	API modular, MongoDB/Mongoose, JWT, bcrypt, roles, validaciones y respuestas HTTP coherentes.	3: completo. 2: mayormente funcional. 1: incompleto o inseguro. 0: no demostrable.

Frontend y gestión de estado	3	Angular, React y Next.js cumplen el propósito definido; rutas, formularios, Hooks y estado global están correctamente aplicados.	3: integración sólida. 2: cumple parcialmente. 1: implementación mínima. 0: no demostrable.
Despliegue, producción y seguridad	2	URLs operativas, variables seguras, HTTPS, CORS/Helmet, controles básicos XSS/CSRF y Lighthouse.	2: completo y verificable. 1: evidencia parcial. 0: no desplegado o sin controles.
GitHub, README y documentación técnica	2	Repositorio ordenado; README completo con URLs, ejecución, evidencias y enlace de YouTube; documentación en /docs.	2: completo. 1: incompleto. 0: ausente o inaccesible.
Video en YouTube con cámaras prendidas	3	Video de 12-15 minutos, enlace en README, participación de todos, cámaras prendidas y explicación técnica clara.	3: cumple íntegramente. 2: omisión menor. 1: participación o evidencia insuficiente. 0: sin video/enlace.
Asistencia y participación	3	Asistencia sostenida, avances verificables y	3: asistencia y participación
Criterio	Puntaje máximo	Logro esperado	Guía de asignación
entre semanas		participación en las clases de construcción del proyecto. Se califica individualmente.	constantes. 2: una ausencia o participación irregular. 1: asistencia limitada. 0: sin evidencia suficiente.
TOTAL	20	Evaluación del producto, proceso y sustentación.	La nota final puede diferenciarse por integrante.

9. Lista de verificación antes de entregar

Fecha y hora límite	
Enlace de entrega en aula virtual	
NRC / sección	
Docente	
Observaciones adicionales	
	Cierre La Evaluación no solicita comenzar un proyecto nuevo. El objetivo es consolidar, integrar, asegurar, desplegar y sustentar el proyecto construido progresivamente durante el curso.

Verificación	Criterio
--------------	----------

☐	El repositorio es accesible y contiene el código final.
☐	El README incluye el enlace de YouTube y todas las URLs desplegadas.
☐	Todos los integrantes aparecen en el video con cámaras prendidas y explican su aporte.
☐	Las aplicaciones desplegadas en producción funcionan desde una ventana de incógnito.
☐	Las credenciales de prueba están documentadas sin exponer secretos reales.
☐	El archivo .env no fue subido; existe .env.example.
☐	La API responde correctamente y MongoDB Atlas permite la conexión del backend desplegado.
☐	Las rutas protegidas y los roles fueron probados.
☐	La documentación técnica y el reporte Lighthouse están en /docs.
☐	Los integrantes revisaron su historial de asistencia y participación.

10. Datos de entrega

11. Registro de calificación por integrante

Este cuadro permite registrar diferencias individuales sustentadas en asistencia, participación, exposición y evidencia de autoría.

Integrante	Aporte / commits	Video y dominio	Asistencia (0-3)	Nota final (0-20)

12. Observaciones del docente

